

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería de Sistemas de Información
Ingeniería en Sistemas de Información y Ciencias de la Computación
Programación I
Ing. David Álvarez, MBA



**Documentación del Proyecto Final - Sistema de Gestión de Ventas e
Inventario para Tienda**

José Daniel Real García 9941-25-837

Plan Fin de Semana

Sección "A"

7 de junio del 2026

1. Introducción y Descripción General

1.1 ¿Qué es el Sistema?

El Sistema de Gestión de Ventas e Inventario para Tienda es una aplicación de consola desarrollada en C++ (estándar C++17) como proyecto final del curso Programación I. Su propósito es administrar de forma integral el inventario de productos y el registro de ventas de una tienda, permitiendo al usuario consultar existencias, procesar transacciones, generar reportes y exportar información, todo con persistencia de datos entre sesiones mediante archivos binarios.

El sistema fue diseñado bajo principios de programación estructurada: funciones modulares bien definidas, estructuras de datos compactas, algoritmos de búsqueda y ordenamiento implementados manualmente, y un manejo robusto de errores mediante bloques try/catch de C++.

1.2 Características Principales

Característica	Descripción
Gestión completa de productos	Registro, listado, búsqueda, actualización de stock y precio, y eliminación lógica.
Proceso de venta interactivo	Selección de múltiples productos, validación de stock en tiempo real y cálculo automático de totales.
Sistema de descuentos y bonos	Descuento por monto (5%) y bono especial por cantidad (3%), más IVA del 12%.
Reportes y ordenamientos	Stock bajo, productos más vendidos, ventas del día, ventas por mes, y ordenamientos por precio y stock.
Persistencia en archivos binarios	Los datos sobreviven entre ejecuciones gracias a productos.dat y ventas.dat.
Exportación de reporte	Genera reporte.txt con inventario completo y resumen de ventas.
Manejo de errores	Entradas no numéricas, valores fuera de rango y archivos corruptos son capturados con try/catch.

1.3 Alcance y Limitaciones

El sistema soporta hasta 200 productos simultáneos en memoria, hasta 500 ventas acumuladas, y hasta 20 productos diferentes por venta. Estas constantes pueden ajustarse modificando las variables globales MAX_PRODUCTOS, MAX_VENTAS y MAX_DETALLES en el archivo main.cpp antes de recompilar.

■ El sistema trabaja exclusivamente en modo consola (terminal). No cuenta con interfaz gráfica. Está diseñado para ejecutarse en Windows, Linux y macOS.

2. Estructura del Proyecto

2.1 Árbol de Archivos

A continuación se muestra la estructura de directorios del proyecto tal como debe encontrarse en el sistema de archivos para su correcto funcionamiento:

```
PROYECTO_FINAL/  
■■■■ main.cpp ← Código fuente principal (único archivo del sistema)  
■■■■ generar_datos_prueba.cpp ← Script auxiliar para inyectar datos de prueba  
■■■■ SistemaTienda.exe ← Ejecutable del sistema (compilado de main.cpp)  
■■■■ GenerarDatos.exe ← Ejecutable auxiliar (compilado de generar_datos_prueba.cpp)  
■■■■ README.md ← Instrucciones de compilación y ejecución  
■■■■ reporte.txt ← Reporte exportado (generado desde el sistema)  
■■■■ evidencias/  
■■■■ productos.dat ← Base de datos binaria de productos  
■■■■ ventas.dat ← Base de datos binaria de ventas
```

2.2 Descripción de Cada Archivo

Archivo	Tipo	Descripción
main.cpp	C++ — Fuente	Contiene la totalidad del código del sistema: estructuras, variables globales, todas las funciones.
generar_datos_prueba.cpp	C++ — Auxiliar	Script independiente que genera los archivos productos.dat y ventas.dat con datos de prueba.
SistemaTienda.exe	Ejecutable	Binario compilado del sistema principal (main.cpp). Es el programa que el usuario ejecuta.
GenerarDatos.exe	Ejecutable	Binario compilado del script generador. Se ejecuta una vez para poblar los archivos .dat.
README.md	Documentación	Guía rápida de compilación, ejecución y descripción de módulos incluida en el proyecto.
reporte.txt	Texto — Salida	Archivo de texto generado por el sistema al usar la opción 'Exportar reporte'. Contiene inventario y resumen de ventas.
evidencias/productos.dat	Binario — BD	Base de datos binaria de productos. Almacena el conteo total de productos seguido del arreglo de structs Producto.
evidencias/ventas.dat	Binario — BD	Base de datos binaria de ventas. Almacena el conteo total de ventas seguido del arreglo de structs Venta.

2.3 Los Archivos .dat — Base de Datos Binaria

Los archivos productos.dat y ventas.dat, ubicados en la subcarpeta evidencias/, constituyen la base de datos persistente del sistema. No son archivos de texto legibles: contienen los datos escritos en formato binario crudo (raw binary), lo que los

hace compactos y eficientes pero no editables manualmente.

Ambos archivos comparten la misma estructura interna:

Estructura de productos.dat:

```
[ int numProductos (4 bytes) ] [ Producto[0] ] [ Producto[1] ] ... [ Producto[n-1] ]
```

Estructura de ventas.dat:

```
[ int numVentas (4 bytes) ] [ Venta[0] ] [ Venta[1] ] ... [ Venta[n-1] ]
```

Al iniciarse el sistema, se leen ambos archivos y se cargan en los arreglos globales `productos[]` y `ventas[]` en memoria RAM. Cualquier operación que modifique datos (registrar, vender, actualizar, eliminar) escribe inmediatamente el arreglo completo de vuelta al archivo, garantizando la persistencia en todo momento.

■ Si los archivos `.dat` no existen al iniciar el programa, el sistema los crea automáticamente vacíos. Si los archivos están corruptos o contienen un número de registros fuera de rango, el sistema los descarta y arranca con datos en blanco, mostrando una advertencia en consola.

■ El script `generar_datos_prueba.cpp` escribe directamente en `evidencias/productos.dat` y `evidencias/ventas.dat`, por lo que debe ejecutarse desde la raíz del proyecto.

3. Estructuras de Datos

El sistema define tres estructuras (struct) que modelan los datos del negocio. Estas estructuras se escriben y leen directamente desde los archivos binarios, por lo que su tamaño en bytes es fijo y predecible.

3.1 struct Producto

```
struct Producto {
    int codigo; // Identificador único (1 - 9999)
    char nombre[60]; // Nombre del producto (máx. 59 chars + \0)
    char categoria[30]; // Categoría (máx. 29 chars + \0)
    double precio; // Precio unitario en Quetzales
    int stock; // Unidades disponibles en bodega
    int totalVendido; // Acumulado histórico de unidades vendidas
    bool activo; // true = activo | false = eliminado lógicamente
};
```

Campo	Tipo	Bytes aprox.	Descripción
codigo	int	4	Código numérico único. No puede repetirse. Rango válido: 1 a 9999.
nombre	char[]	60	Nombre del producto. Cadena terminada en \0. Máximo 59 caracteres útiles.
categoria	char[]	30	Categoría a la que pertenece el producto (Bebidas, Granos, Higiene, etc.).
precio	double	8	Precio unitario en Quetzales. Mínimo 0.01. Almacenado con precisión de doble.
stock	int	4	Unidades disponibles. Se decrementa en cada venta y puede actualizarse manualmente.
totalVendido	int	4	Contador acumulado de unidades vendidas. Se incrementa con cada venta confirmada.
activo	bool	1	Bandera de eliminación lógica. false = producto eliminado (oculto en listados).

3.2 struct DetalleVenta

Representa un renglón dentro de una venta: qué producto, cuántas unidades y a qué precio se vendieron. Cada venta puede contener hasta 20 detalles.

```
struct DetalleVenta {
    int codigoProducto; // Referencia al código del producto vendido
    char nombreProducto[60]; // Snapshot del nombre al momento de la venta
    int cantidad; // Unidades vendidas de este producto
    double precioUnitario; // Precio al momento de la venta (snapshot)
    double subtotalDetalle; // cantidad x precioUnitario (redondeado a 2 decimales)
```

```
};
```

■ El nombre y precio del producto se guardan como 'snapshot' (copia al momento de la venta) para que los reportes históricos sean precisos aunque el producto cambie de precio o nombre.

3.3 struct Venta

```
struct Venta {  
    int idVenta; // ID incremental (numVentas + 1 al crear)  
    int dia, mes, anio; // Fecha del sistema al momento de la venta  
    int hora, minuto; // Hora del sistema al momento de la venta  
    DetalleVenta detalles[20]; // Arreglo de hasta 20 líneas de detalle  
    int numDetalles; // Cuántos detalles tiene esta venta (1-20)  
    double subtotal; // Suma de subtotales de todos los detalles  
    double descuento; // Descuento calculado (monto + bono)  
    double iva; // IVA 12% sobre la base gravable  
    double total; // Monto final a cobrar al cliente  
    int totalArticulos; // Suma de todas las cantidades de la venta  
};
```

El campo `idVenta` se asigna como `numVentas + 1` al momento de crear la venta, garantizando un identificador incremental único. La fecha y hora se capturan automáticamente del reloj del sistema usando las funciones `time()` y `localtime()` de la biblioteca `<ctime>`.

4. Constantes y Variables Globales

4.1 Constantes del Sistema

Constante	Valor	Significado
ARCHIVO_PRODUCTOS	"productos.dat"	Nombre/ruta del archivo binario de productos.
ARCHIVO_VENTAS	"ventas.dat"	Nombre/ruta del archivo binario de ventas.
ARCHIVO_REPORTE	"reporte.txt"	Nombre del archivo de texto exportado.
MAX_PRODUCTOS	200	Capacidad máxima de productos en memoria.
MAX_DETALLES	20	Máximo de líneas de producto por venta.
MAX_VENTAS	500	Capacidad máxima de ventas en memoria.
IVA	0.12	Tasa de IVA aplicada (12%).
VERSION	"v1.0"	Versión del sistema mostrada en la cabecera.

4.2 Variables Globales

```
Producto productos[MAX_PRODUCTOS]; // Arreglo principal de productos en RAM
int numProductos = 0; // Cantidad actual de productos cargados

Venta ventas[MAX_VENTAS]; // Arreglo principal de ventas en RAM
int numVentas = 0; // Cantidad actual de ventas cargadas

double ventasPorMes[12][31]; // Matriz de ventas: [mes 0-11][día 0-30]
```

La matriz `ventasPorMes[12][31]` es una estructura bidimensional que acumula el monto total vendido para cada combinación de mes y día del año. Se reconstruye cada vez que se necesita (función `cargarMatrizVentas()`), recorriendo el arreglo de ventas e indexando con `mes-1` y `día-1` para adaptar los índices a base cero.

5. Funciones Principales — Explicación Detallada

5.1 Utilidades de Entrada Segura

Estas funciones garantizan que el usuario nunca pueda ingresar datos inválidos. Todas utilizan try/catch internamente y repiten la solicitud hasta recibir un valor válido.

leerEnteroSeguro(mensaje, minVal, maxVal) → int

Lee una línea completa con `getline()`, la convierte con `stoi()` y valida que no queden caracteres no numéricos (verificando `pos == entrada.size()`). Si el valor está fuera del rango `[minVal, maxVal]` lanza `out_of_range`. El ciclo `while(true)` no termina hasta que el usuario ingrese un entero válido en el rango especificado.

leerDoubleSeguro(mensaje, minVal) → double

Análogo a `leerEnteroSeguro` pero para números de punto flotante, usando `stod()`. Valida que el valor sea mayor o igual a `minVal` (generalmente 0.01 para precios).

leerStringSeguro(mensaje, maxLen) → string

Lee una cadena no vacía con longitud máxima `maxLen`. Lanza `invalid_argument` si está vacía y `length_error` si supera el límite.

5.2 Manejo de Archivos

cargarProductos() — void

```
void cargarProductos() {
    ifstream archivo(ARCHIVO_PRODUCTOS, ios::binary);
    // Lee primero el int numProductos
    archivo.read(reinterpret_cast(&numProductos;), sizeof(int));
    // Valida rango para detectar archivos corruptos
    if (numProductos < 0 || numProductos > MAX_PRODUCTOS)
        throw runtime_error("Archivo de productos corrupto.");
    // Lee el bloque completo de estructuras Producto
    archivo.read(reinterpret_cast(productos), sizeof(Producto) * numProductos);
}
```

Si el archivo no existe, el sistema continúa con `numProductos = 0` sin lanzar error. El uso de `reinterpret_cast<char*>` permite leer la estructura binaria byte a byte directamente hacia la memoria del arreglo, lo cual es correcto siempre que el mismo compilador que escribió el archivo también lo lea (mismo layout de memoria).

guardarProductos() — void

Escribe el archivo `productos.dat` desde cero (`ios::trunc`) cada vez que se llama. Primero escribe el entero `numProductos` y luego el bloque completo del arreglo `productos[]`. Se llama después de cualquier operación que modifique el inventario.

guardarVenta(const Venta& v) — void

Agrega la nueva venta al arreglo en memoria (`ventas[numVentas++] = v`) y luego reescribe el archivo completo `ventas.dat`. También actualiza el stock y `totalVendido` de los productos involucrados antes de llamar a `guardarProductos()`.

cargarMatrizVentas() — void

Inicializa la matriz `ventasPorMes[12][31]` a cero y la recorre reconstruyéndola: para cada venta, usa `mes-1` y `dia-1` como índices y acumula el total. No lee archivos; trabaja exclusivamente sobre el arreglo `ventas[]` ya cargado en memoria.

5.3 Módulo 1 — Gestión de Productos

encontrarProductoPorCodigo(int codigo) → int

Realiza una búsqueda secuencial sobre el arreglo `productos[]`. Retorna el índice del primer producto activo cuyo campo `codigo` coincida, o `-1` si no existe. Es la función auxiliar más utilizada en el sistema, pues casi todas las operaciones necesitan localizar un producto antes de modificarlo.

registrarProducto() — void

Solicita todos los campos de un nuevo producto. Valida que el código sea único llamando a `encontrarProductoPorCodigo()` en un bucle. Inicializa `totalVendido = 0` y `activo = true`. Al confirmar, agrega el producto al arreglo y llama a `guardarProductos()`.

eliminarProducto() — void

Implementa eliminación lógica: no borra el registro del arreglo ni del archivo, sino que asigna `activo = false`. Los productos inactivos no aparecen en listados, búsquedas ni ventas, pero conservan sus datos históricos de ventas.

buscarProductoPorNombre() — void

Convierte tanto la búsqueda del usuario como el nombre de cada producto a minúsculas usando `std::transform` con `::tolower`, luego aplica `string::find()` para encontrar coincidencias parciales. Esto permite buscar 'coca' y encontrar 'Coca Cola 600ml'.

5.4 Módulo 2 — Proceso de Venta

crearVenta() — void

Función central del módulo de ventas. Crea un objeto `Venta v` y entra en un ciclo `do-while` donde el usuario puede agregar productos. Por cada producto:

- Muestra la lista de productos activos y solicita el código.
- Verifica que el producto exista y tenga stock disponible.
- Solicita la cantidad (máximo: stock disponible).
- Calcula el subtotal del detalle con redondeo a 2 decimales.
- Descuenta el stock y suma a `totalVendido` del producto en memoria.
- Agrega el `DetalleVenta` al arreglo `v.detalles[]`.

Al finalizar la carga de productos, calcula descuentos e IVA, muestra el resumen completo y solicita confirmación. Si el usuario cancela, el stock se revierte para todos los productos agregados antes de retornar.

calcularDescuento(double subtotal, int totalArticulos) → double

Calcula el descuento total aplicable según dos reglas de negocio independientes que pueden combinarse:

- Descuento por monto: si el subtotal supera Q500.00, se aplica un 5% de descuento sobre el subtotal.
- Bono especial por cantidad: si el total de artículos vendidos es múltiplo de 5 (y mayor que 0), se aplica un 3% adicional también sobre el subtotal.

Ambos descuentos se acumulan y se muestran en consola al aplicarse. El valor retornado es el descuento total en Quetzales.

mostrarResumenVenta(const Venta& v) — void

Imprime en pantalla la tabla de detalles de la venta con formato alineado usando setw() e iomanip, seguido del desglose financiero completo: subtotal, descuento, IVA y total a pagar. Se llama tanto al confirmar la venta como al previsualizar.

5.5 Módulo 3 — Reportes y Ordenamientos

Función	Algoritmo	Criterio	Orden
ordenarProductosPorPrecio(bool asc)	Bubble Sort	precio	Asc. o Desc.
ordenarProductosPorStock(bool asc)	Selection Sort	stock	Asc. o Desc.
ordenarProductosPorVentas()	Insertion Sort	totalVendido	Descendente

Los algoritmos trabajan sobre un arreglo auxiliar de índices (no sobre el arreglo principal), lo que evita copiar estructuras grandes y preserva el orden original de productos[] sin alterarlo. Solo se muestran en pantalla; no modifican los datos.

reporteStockBajo() — void

Solicita al usuario un umbral numérico y filtra todos los productos activos cuyo stock sea menor o igual a dicho umbral. Es especialmente útil para identificar productos que necesitan reposición antes de agotarse.

reporteVentasDia() — void

Captura la fecha actual del sistema con time() y localtime(), y recorre el arreglo de ventas filtrando aquellas cuya fecha (día, mes, año) coincida con hoy. Muestra cada venta con su hora, artículos y total, y al final suma el acumulado del día.

reporteVentasPorMes() — void

Llama a cargarMatrizVentas() para reconstruir ventasPorMes[], luego recorre las 12 filas (meses) e imprime los días con ventas registradas y el total mensual. Solo muestra meses que tengan al menos una venta.

5.6 Módulo 4 — Utilidades

exportarReporteTxt() — void

Crea el archivo reporte.txt con tres secciones: inventario de todos los productos activos (con código, nombre, precio, stock y total vendido), resumen de ventas (cantidad total y monto acumulado) y ventas desglosadas por mes. Usa ofstream con formato setw() para lograr columnas alineadas.

reiniciarInventario() — void

Muestra una advertencia y exige que el usuario escriba exactamente la palabra CONFIRMAR (sensible a mayúsculas) antes de proceder. Al confirmar, pone numProductos = 0 y numVentas = 0, y sobrescribe ambos archivos .dat con contadores en cero. Esta operación es irreversible.

mostrarEstadisticasGenerales() — void

Recorre el arreglo de productos calculando: total de productos activos, total de productos agotados (stock == 0), valor total del inventario (precio × stock de cada producto), producto más caro y más barato, y total de artículos vendidos históricamente. Para las ventas calcula el monto total acumulado y el promedio por venta.

6. Lógica Comercial y Fórmulas de Venta

El sistema implementa una lógica de precios completa que incluye descuentos automáticos y la aplicación del Impuesto al Valor Agregado (IVA). A continuación se explica cada concepto y su fórmula exacta:

6.1 Subtotal de Detalle

```
subtotalDetalle = round(precioUnitario × cantidad × 100.0) / 100.0
```

Para cada línea de producto, el subtotal se calcula multiplicando el precio unitario por la cantidad solicitada. El resultado se redondea a exactamente 2 decimales usando la técnica $\text{round}(x \times 100) / 100$ para evitar errores de punto flotante.

6.2 Subtotal General

```
subtotal = Σ subtotalDetalle[i] para i = 0 hasta numDetalles-1
```

El subtotal general de la venta es la suma de todos los subtotales de detalle.

6.3 Descuento por Monto

```
Si (subtotal > 500.00): descuento_monto = subtotal × 0.05 (5%) Sino: descuento_monto = 0.00
```

Este descuento se activa automáticamente cuando el subtotal de la venta supera los Q500.00. El sistema informa en pantalla cuando lo aplica.

6.4 Bono Especial por Cantidad

```
Si (totalArticulos % 5 == 0 AND totalArticulos > 0): bono_cantidad = subtotal × 0.03 (3%) Sino:  
bono_cantidad = 0.00
```

Este bono premia a los clientes que compran un total de artículos cuya cantidad sea múltiplo exacto de 5 (5, 10, 15, 20, ...). Es independiente del descuento por monto; ambos pueden aplicarse simultáneamente.

6.5 Descuento Total

```
descuento = descuento_monto + bono_cantidad
```

6.6 Base Gravable

```
base_gravable = subtotal - descuento
```

La base gravable es el monto sobre el cual se calcula el IVA. Los descuentos se aplican antes de calcular el impuesto, lo que es beneficioso para el cliente.

6.7 IVA (12%)

```
iva = round(base_gravable × 0.12 × 100.0) / 100.0
```

El IVA se calcula sobre la base gravable (subtotal menos descuentos) al 12%, en concordancia con la tasa vigente en Guatemala. Se redondea a 2 decimales.

6.8 Total Final

```
total = round((base_gravable + iva) × 100.0) / 100.0
```

El total final a cobrar es la suma de la base gravable más el IVA. Es el único valor que se presenta al cliente como 'TOTAL A PAGAR'.

6.9 Ejemplo Práctico

Suponga una venta con 10 unidades de Coca Cola a Q7.50 y 3 unidades de Pan Bimbo a Q15.00:

Concepto	Cálculo	Resultado
Subtotal Coca Cola	$10 \times Q7.50$	Q75.00
Subtotal Pan Bimbo	$3 \times Q15.00$	Q45.00
Subtotal General	$Q75.00 + Q45.00$	Q120.00
Total Artículos	$10 + 3$	13 (no múltiplo de 5)
Descuento por monto	$Q120.00 < Q500 \rightarrow$ no aplica	Q0.00
Bono especial	$13 \% 5 \neq 0 \rightarrow$ no aplica	Q0.00
Descuento Total	$Q0.00 + Q0.00$	Q0.00
Base Gravable	$Q120.00 - Q0.00$	Q120.00
IVA (12%)	$Q120.00 \times 0.12$	Q14.40
TOTAL A PAGAR	$Q120.00 + Q14.40$	Q134.40

Este ejemplo corresponde exactamente a la venta de prueba incluida en el archivo `ventas.dat` del proyecto y coincide con los datos mostrados en el reporte generado (`reporte.txt`).

7. Algoritmos de Búsqueda y Ordenamiento

7.1 Búsqueda Secuencial por Código

```
int encontrarProductoPorCodigo(int codigo) {
    for (int i = 0; i < numProductos; i++) {
        if (productos[i].codigo == codigo && productos[i].activo)
            return i; // Retorno inmediato al primer match
    }
    return -1; // No encontrado
}
```

Complejidad temporal: $O(n)$ en el peor caso. Para el tamaño máximo de 200 productos esto es completamente adecuado. Solo considera productos activos.

7.2 Búsqueda Parcial por Nombre (case-insensitive)

```
// Convierte búsqueda y nombre a minúsculas, luego usa string::find()
transform(busqLow.begin(), busqLow.end(), busqLow.begin(), ::tolower);
// ...
if (nombreLow.find(busqLow) != string::npos) { /* coincidencia */ }
```

Permite búsquedas parciales insensibles a mayúsculas/minúsculas. Devuelve todos los productos que contengan la cadena buscada en cualquier parte de su nombre.

7.3 Bubble Sort — Ordenar por Precio

```
// Trabaja sobre arreglo auxiliar de índices, no sobre productos[] directamente
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        bool intercambiar = ascendente
        ? productos[indices[j]].precio > productos[indices[j+1]].precio
        : productos[indices[j]].precio < productos[indices[j+1]].precio;
        if (intercambiar) swap(indices[j], indices[j+1]);
    }
}
```

Complejidad: $O(n^2)$. Cada pasada garantiza que el elemento más grande (o pequeño) quede en su posición final. El parámetro booleano ascendente invierte la comparación para soporte de ambas direcciones.

7.4 Selection Sort — Ordenar por Stock

```
for (int i = 0; i < n - 1; i++) {
    int seleccionado = i;
    for (int j = i + 1; j < n; j++) {
        bool esMinimo = ascendente
        ? productos[indices[j]].stock < productos[indices[selectedo]].stock
```

```
: productos[indices[j]].stock > productos[indices[seleccionado]].stock;
if (esMinimo) seleccionado = j;
}
if (seleccionado != i) swap(indices[i], indices[seleccionado]);
}
```

Complejidad: $O(n^2)$. En cada iteración externa busca el mínimo (o máximo) del subarreglo restante y lo coloca en su posición. Realiza como máximo $n-1$ intercambios (ventaja respecto a Bubble Sort en número de swaps).

7.5 Insertion Sort — Ordenar por Ventas Acumuladas

```
// Descendente por totalVendido (mayor a menor)
for (int i = 1; i < n; i++) {
    int key = indices[i];
    int j = i - 1;
    while (j >= 0 && productos[indices[j]].totalVendido < productos[key].totalVendido) {
        indices[j + 1] = indices[j];
        j--;
    }
    indices[j + 1] = key;
}
```

Complejidad: $O(n^2)$ peor caso, $O(n)$ mejor caso (arreglo ya ordenado). Inserta cada elemento en su posición correcta dentro de la parte ya ordenada, desplazando los demás. Útil para mostrar el ranking de productos más vendidos.

8. Guía Paso a Paso — Compilar y Ejecutar

8.1 Requisitos Previos

Sistema Operativo	Requisito	Cómo obtenerlo
Windows	MinGW-w64 con g++ (C++17+)	https://www.mingw-w64.org/downloads/ — Instalar y agregar al PATH
Linux (Ubuntu/Debian)	g++ incluido o instalar	<code>sudo apt update && sudo apt install g++ build-essential</code>
macOS	Xcode Command Line Tools	<code>xcode-select --install</code> (en Terminal)

■ Para verificar que g++ está instalado y disponible, abra una terminal y ejecute: `g++ --version`. Debería mostrar la versión instalada (se recomienda g++ 11 o superior).

8.2 Paso 1 — Abrir la Terminal en la Carpeta del Proyecto

- Windows: Abra PowerShell o CMD. Navegue con `cd C:\ruta\al\proyecto`.
- Linux/macOS: Abra la terminal. Navegue con `cd /ruta/al/proyecto`.
- VS Code: Abra la carpeta del proyecto (Archivo → Abrir Carpeta) y presione `Ctrl+`` para abrir la terminal integrada.

8.3 Paso 2 — Generar Datos de Prueba (Primera Ejecución)

Este paso es opcional pero recomendado para la primera ejecución. El script genera 8 productos de ejemplo y una venta de prueba en los archivos binarios.

```
# Compilar el generador de datos
g++ generar_datos_prueba.cpp -o GenerarDatos

# Ejecutar en Windows
.\GenerarDatos.exe

# Ejecutar en Linux / macOS
./GenerarDatos
```

Al ejecutarse, el programa confirmará en pantalla: `[OK] Archivos productos.dat y ventas.dat generados con datos de prueba. 8 productos | 1 venta de prueba`

■ Los archivos se crean en la subcarpeta `evidencias/`. Asegúrese de que dicha carpeta exista antes de ejecutar el generador, o créela manualmente con: `mkdir evidencias`

8.4 Paso 3 — Compilar el Sistema Principal

```
# Compilar con C++17 y advertencias activas
g++ -std=c++17 -Wall main.cpp -o SistemaTienda
```

Parámetro	Significado
<code>-std=c++17</code>	Usa el estándar C++17 (necesario para algunas funciones de y strings modernas).

Parámetro	Significado
-Wall	Activa todos los avisos del compilador para detectar posibles problemas en el código.
main.cpp	Archivo fuente de entrada.
-o SistemaTienda	Nombre del ejecutable de salida (en Windows genera SistemaTienda.exe automáticamente).

8.5 Paso 4 — Ejecutar el Sistema

```
# Windows
.\SistemaTienda.exe

# Linux / macOS
./SistemaTienda
```

Al iniciar, el sistema muestra la pantalla de bienvenida con los datos del autor, carga automáticamente los archivos productos.dat y ventas.dat e informa cuántos registros fueron encontrados.

```
#####
# SISTEMA DE GESTIÓN DE VENTAS E INVENTARIO #
# Proyecto Final C++ #
# #
# Autor : José Daniel Real García #
# Carnet: 9941 - 25 - 837 #
# Curso : Programación I #
# Año : 2026 #
#####
Cargando datos del sistema...
8 producto(s) cargado(s).
1 venta(s) cargada(s).
```

8.6 Resumen de Comandos por Sistema Operativo

Acción	Windows (PowerShell/CMD)	Linux / macOS (Bash)
Compilar generador	g++ generar_datos_prueba.cpp -o GenerarDatos	g++ generar_datos_prueba.cpp -o GenerarDatos
Ejecutar generador	.\GenerarDatos.exe	./GenerarDatos
Compilar sistema	g++ -std=c++17 -Wall main.cpp -o SistemaTienda	g++ -std=c++17 -Wall main.cpp -o SistemaTienda
Ejecutar sistema	.\SistemaTienda.exe	./SistemaTienda
Crear carpeta	mkdir evidencias	mkdir -p evidencias

9. Navegación del Sistema — Menús Disponibles

9.1 Menú Principal

Opción	Módulo / Acción
[1]	Módulo 1 — Gestión de Productos
[2]	Módulo 2 — Proceso de Venta
[3]	Módulo 3 — Reportes
[4]	Módulo 4 — Utilidades del Sistema
[0]	Salir del sistema

9.2 Módulo 1 — Gestión de Productos

Opción	Función	Descripción
[1]	Registrar producto	Agrega un nuevo producto con código único, nombre, categoría, precio y stock inicial.
[2]	Listar productos	Muestra tabla de todos los productos activos con código, nombre, categoría, precio, stock y ventas acumuladas.
[3]	Buscar por código	Búsqueda exacta por código numérico. Muestra todos los detalles del producto.
[4]	Buscar por nombre	Búsqueda parcial insensible a mayúsculas. Devuelve todos los productos que coincidan.
[5]	Actualizar stock	Modifica directamente la cantidad de unidades disponibles de un producto.
[6]	Modificar precio	Cambia el precio unitario de un producto. Los cambios aplican a ventas futuras.
[7]	Eliminar producto	Desactiva lógicamente el producto (activo = false). No se elimina del archivo.
[0]	Volver	Regresa al menú principal.

9.3 Módulo 3 — Reportes

Opción	Reporte / Orden
[1]	Productos con menor stock (filtro por umbral definido por el usuario)
[2]	Productos más vendidos (Insertion Sort descendente por totalVendido)
[3]	Ventas totales del día (filtradas por fecha actual del sistema)
[4]	Ventas por mes, desglosadas por día (usando la matriz ventasPorMes[12][31])
[5]	Ordenar por precio — Ascendente (Bubble Sort)

Opción	Reporte / Orden
[6]	Ordenar por precio — Descendente (Bubble Sort)
[7]	Ordenar por stock — Ascendente (Selection Sort)
[8]	Ordenar por stock — Descendente (Selection Sort)
[9]	Ordenar por ventas acumuladas — Descendente (Insertion Sort)

10. Manejo de Errores con try/catch

El sistema implementa manejo estructurado de excepciones en C++ mediante bloques try/catch en todas las situaciones críticas. Esto impide que el programa se cierre inesperadamente ante entradas incorrectas o problemas de archivos.

Situación	Excepción	Comportamiento del sistema
Entrada no numérica (ej: escribir 'abc' donde se espera un número)	invalid_argument	Muestra [ERROR] y vuelve a solicitar el dato.
Valor fuera del rango válido (ej: opción 9 en menú de 0-4)	out_of_range	Muestra [ERROR] con el rango aceptado y vuelve a solicitar.
Texto vacío donde se requiere un nombre	invalid_argument	Muestra [ERROR] y vuelve a solicitar.
Texto demasiado largo (supera maxLen)	length_error	Muestra [ERROR] con el máximo permitido.
Archivo .dat no encontrado al iniciar	runtime_error (capturado)	El sistema arranca con 0 registros sin interrumpir la ejecución.
Archivo .dat con número de registros inválido (corrupto)	runtime_error	Descarta el archivo y arranca con 0 registros, mostrando [ADVERTENCIA].
No se puede crear/escribir el archivo .dat o reporte.txt	runtime_error	Muestra [ERROR] con el motivo y continúa el programa.

■ ■ Importante: Las funciones de entrada segura (leerEnteroSeguro, leerDoubleSeguro, leerStringSeguro) nunca retornan un valor inválido. El control de flujo no avanza hasta que el usuario ingrese un dato correcto.

11. Datos de Prueba Incluidos

El script generar_datos_prueba.cpp inyecta los siguientes registros en los archivos .dat para permitir probar el sistema sin necesidad de registrar productos manualmente:

11.1 Productos de Prueba (8 productos)

Cód.	Nombre	Categoría	Precio	Stock	Vendido
101	Coca Cola 600ml	Bebidas	Q7.50	90	10
102	Pan Bimbo Grande	Panificacion	Q15.00	47	3
103	Aceite Ideal 1L	Aceites	Q28.50	30	0
104	Arroz Toro 1LB	Granos	Q5.00	200	0
105	Frijoles Negros 1LB	Granos	Q6.50	150	0
201	Jabón Palmolive	Higiene	Q9.75	60	0
202	Shampoo Head Shoulders	Higiene	Q35.00	25	0
301	Papel Higiénico Suave	Hogar	Q22.00	5	0

11.2 Venta de Prueba (1 venta registrada)

El generador también crea una venta de prueba (ID #1) con fecha 06/06/2026 a las 10:30:

Campo	Valor
ID de Venta	#1
Fecha	06/06/2026 10:30
Producto 1	Coca Cola 600ml — 10 unidades × Q7.50 = Q75.00
Producto 2	Pan Bimbo Grande — 3 unidades × Q15.00 = Q45.00
Total Artículos	13 (10 + 3)
Subtotal	Q120.00
Descuento	Q0.00 (subtotal < Q500 y 13 no es múltiplo de 5)
IVA (12%)	Q14.40
TOTAL	Q134.40

■ El reporte.txt adjunto al proyecto muestra datos levemente distintos (stock y ventas acumuladas diferentes) porque fue generado tras una sesión de prueba real en la que se realizaron ventas adicionales. El reporte es una exportación instantánea del estado del sistema en ese momento.

12. Notas Técnicas y Consideraciones Importantes

12.1 Compatibilidad de Archivos Binarios

Los archivos `productos.dat` y `ventas.dat` son compatibles únicamente con la versión del compilador y arquitectura con la que fueron generados. Si se recompila el sistema en una plataforma diferente (ej: de Windows x64 a Linux x32), el layout de memoria de las estructuras puede cambiar y los archivos existentes volverán a interpretarse incorrectamente. En ese caso se deben regenerar con el script auxiliar.

12.2 Codificación de Caracteres — Windows

En Windows, el sistema ejecuta automáticamente `system("chcp 65001 > nul")` al inicio para configurar la consola en UTF-8, permitiendo mostrar caracteres especiales del español (tildes, ñ, etc.) correctamente. En Linux y macOS, UTF-8 es el estándar y no requiere configuración adicional.

12.3 Redondeo de Valores Monetarios

Todos los cálculos monetarios usan la técnica $\text{round}(\text{valor} \times 100.0) / 100.0$ para evitar errores de acumulación de punto flotante (ej: $0.1 + 0.2$ en IEEE 754 no es exactamente 0.3). Esto garantiza que los totales mostrados y guardados en los archivos siempre tengan exactamente 2 decimales significativos.

12.4 Modificar los Límites del Sistema

Si el negocio crece y se necesitan más productos o ventas, basta con modificar las constantes globales en `main.cpp` y recompilar:

```
const int MAX_PRODUCTOS = 200; // Cambiar a 500, 1000, etc.
const int MAX_DETALLES = 20; // Máximo de productos por venta
const int MAX_VENTAS = 500; // Cambiar a 2000, etc.
```

■ Al cambiar `MAX_PRODUCTOS` o `MAX_VENTAS` y recompilar, los archivos `.dat` existentes deben eliminarse y regenerarse con el script auxiliar, ya que el tamaño de las estructuras guardadas habrá cambiado.

12.5 Librerías Utilizadas

Librería	Uso principal
<code><iostream></code>	Entrada/salida estándar por consola (<code>cin</code> , <code>cout</code> , <code>cerr</code>).
<code><fstream></code>	Manejo de archivos binarios y de texto (<code>ifstream</code> , <code>ofstream</code>).
<code><string></code>	Manejo de cadenas <code>std::string</code> .
<code><cmath></code>	Función <code>round()</code> para redondeo monetario.
<code><ctime></code>	Obtener fecha y hora del sistema (<code>time()</code> , <code>localtime()</code>).
<code><iomanip></code>	Formato de salida alineada (<code>setw()</code> , <code>setfill()</code> , <code>setprecision()</code> , <code>fixed</code>).
<code><limits></code>	<code>numeric_limits</code> para ignorar buffer del <code>cin</code> correctamente.

Librería	Uso principal
<algorithm>	std::transform (conversión a minúsculas) y std::swap (ordenamientos).
<cstring>	strncpy para copiar cadenas char[] de forma segura.
<stdexcept>	Clases de excepción estándar (invalid_argument, out_of_range, etc.).

13. Resumen Ejecutivo del Proyecto

Aspecto	Detalle
Nombre del proyecto	Sistema de Gestión de Ventas e Inventario para Tienda
Autor	José Daniel Real García
Carnet	9941 - 25 - 837
Curso	Programación I
Año académico	2026
Lenguaje	C++17
Compilador recomendado	g++ 11 o superior (MinGW-w64 en Windows)
Tipo de aplicación	Consola (CLI) — Sin interfaz gráfica
Persistencia de datos	Archivos binarios: productos.dat y ventas.dat (en carpeta evidencias/)
Módulos implementados	4 módulos: Productos, Ventas, Reportes, Utilidades
Capacidad máxima	200 productos 500 ventas 20 artículos por venta
Algoritmos incluidos	Bubble Sort, Selection Sort, Insertion Sort + Búsqueda secuencial
Manejo de errores	try/catch en entradas de usuario y operaciones de archivo
Lógica comercial	IVA 12%, descuento 5% (>Q500), bono 3% (artículos múltiplo de 5)
Compatibilidad	Windows, Linux, macOS

Este sistema demuestra el dominio de los conceptos fundamentales de Programación I: estructuras de datos, arreglos, funciones modulares, algoritmos de búsqueda y ordenamiento, manejo de archivos binarios, y manejo de excepciones en C++. El código está organizado en un único archivo fuente (main.cpp) con más de 1,270 líneas, completamente comentado y listo para compilar.

— José Daniel Real García · Carnet: 9941-25-837 · Programación I · 2026 —